MAULANA ABUL KALAM AZAD UNIVERSITY OF TECHNOLOGY, WEST BENGAL

Paper Code: ESCS201 — Programming for Problem Solving
UPID: 002008 | Sem-2 | 2022-2023

COMPLETE ANSWER KEY

GROUP-A — Very Short Answer Type Questions [1 × 10 = 10]

(I) Use of continue in a loop

The **continue** statement skips the remaining statements in the current iteration of a loop and immediately jumps to the next iteration. It does not terminate the loop; only the current cycle is skipped.

(II) String function to reverse a string

strrev() (declared in <string.h>) reverses a string in place. Example: *strrev(str);* — modifies *str* so 'hello' becomes 'olleh'. (In standard C99/C11, strrev is non-standard but widely available on Windows compilers; on GCC/Linux a manual loop or custom function is used.)

(III) Time complexity of Bubble Sort

Worst & Average case: O(n²) — two nested loops each run up to n times. **Best case: O(n)** — when the array is already sorted and an optimised version with an early-exit flag is used.

(IV) Preprocessors

Preprocessors are directives processed by the C preprocessor *before* compilation begins. They start with #. Common examples: **#include** (file inclusion), **#define** (macro definition), **#ifdef / #ifndef** (conditional compilation). They do not generate machine code directly; they transform the source text.

(V) Data structure used by Recursion

Recursion internally uses a **Stack** (the call stack). Each recursive call pushes a new stack frame (containing local variables and return address) onto the stack; when a call returns the frame is popped.

(VI) Header file for string operations

<string.h> is the header file that provides string-handling functions such as strlen(), strcpy(), strcat(), strcmp(), strrev(), etc.

(VII) Nested if-else

A **nested if-else** is an if-else statement placed inside another if or else block, allowing multi-way decisions. Example: *if(a>b){ if(a>c) … else … } else { … }*

(VIII) Use of goto statement

The **goto** statement provides an unconditional jump to a labelled statement elsewhere in the same function. Syntax: *goto label;* … *label: statement;*. It is generally discouraged as it makes code harder to read and maintain.

(IX) Linear searching in an array

Linear (sequential) search checks each element of an array one by one from the beginning until the target element is found or the array ends. Time complexity: **O(n)**. It works on both sorted and unsorted arrays.

(X) Macro

A **Macro** is a fragment of code given a name using **#define**. Wherever the name appears, the preprocessor substitutes the defined text before compilation. Example: *#define PI 3.14159* — every occurrence of PI is replaced by 3.14159. Macros can also take arguments (function-like macros).

(XI) Value of printf("%d", --a) when a = 2

Output: 1

The pre-decrement operator **--a** decrements *a* from 2 to 1 *before* the value is used in printf. So 1 is printed. After the statement, a = 1.

(XII) goto statement in C

The **goto** statement is a control flow statement that transfers program execution unconditionally to a specified label within the same function. Syntax:

goto label_name;

label_name: // code

It is rarely used in modern C programming.

GROUP-B — Short Answer Type Questions [5 × 3 = 15]

Q2. Structure & Array of Structures

What is a Structure?

A **structure** is a user-defined data type in C that groups variables of different data types under a single name. Each variable inside a structure is called a *member* or *field*. Keyword: **struct**.

What is an Array of Structures?

An **array of structures** is a collection of multiple structure variables of the same type, stored contiguously in memory. It is used to handle a group of records (e.g., multiple students).

```
#include <stdio.h>
struct Student {
    char name[30];
    int roll;
    float marks;
};

int main() {
    struct Student s[3] = {
        {"Alice", 1, 88.5},
        {"Bob",   2, 76.0},
        {"Carol", 3, 91.0}
    };
    for (int i = 0; i < 3; i++)
        printf("Name: %s, Roll: %d, Marks: %.1f\n",
               s[i].name, s[i].roll, s[i].marks);
    return 0;
}
```

Q3. Program to find roots of a Quadratic Equation

A quadratic equation $ax^2 + bx + c = 0$ has discriminant $D = b^2 - 4ac$. If $D > 0$: two real roots; $D = 0$: one repeated root; $D < 0$: complex roots.

```
#include <stdio.h>
#include <math.h>

int main() {
    float a, b, c, D, r1, r2;
    printf("Enter a, b, c: ");
    scanf("%f %f %f", &a, &b, &c);

    D = b*b - 4*a*c;

    if (D > 0) {
        r1 = (-b + sqrt(D)) / (2*a);
        r2 = (-b - sqrt(D)) / (2*a);
        printf("Two real roots: %.2f and %.2f\n", r1, r2);
    } else if (D == 0) {
        r1 = -b / (2*a);
        printf("Repeated root: %.2f\n", r1);
    } else {
        float real = -b / (2*a);
        float imag = sqrt(-D) / (2*a);
```

```
            printf("Complex roots: %.2f + %.2fi and %.2f - %.2fi\n",
                   real, imag, real, imag);
        }
        return 0;
    }
```

Q4. Flowchart to evaluate Factorial of a number

The following steps describe the flowchart logic for computing n!:

START → Input n → Set fact = 1, i = 1 → **Decision:** Is i <= n? → **Yes:** fact = fact × i, i = i + 1 → loop back → **No:** Print fact → **STOP**

Equivalent C code for verification:

```
#include <stdio.h>
int main() {
    int n, i, fact = 1;
    printf("Enter n: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++)
        fact *= i;
    printf("Factorial = %d\n", fact);
    return 0;
}
```

Q5. Conditional Operator

The **conditional (ternary) operator ? :** is the only ternary operator in C. It evaluates a condition and returns one of two values.

Syntax: condition ? expression_if_true : expression_if_false

```
#include <stdio.h>
int main() {
    int a = 10, b = 20;
    int max = (a > b) ? a : b;   // max = 20
    printf("Max = %d\n", max);

    // Equivalent if-else:
    // if (a > b) max = a; else max = b;
    return 0;
}
```

Q6. Output of the given code

```
#include <stdio.h>
int main() {
    int a[5] = {1, 2, 3, 4, 5};
    int i;
    for (i = 0; i < 5; i++)
        if ((char)a[i] == '5')
            printf("%d\n", a[i]);
        else
            printf("FAIL\n");
}
```

Output:

```
FAIL
```

```
FAIL
FAIL
FAIL
FAIL
```

Explanation: The character literal **'5'** has ASCII value **53**. The array elements are integers 1–5. When cast to char, they become ASCII values 1, 2, 3, 4, 5 — none of which equals 53 ('5'). Therefore the condition is always false and 'FAIL' is printed five times.

GROUP-C — Long Answer Type Questions [15 × 3 = 45]

Q7(a). Difference between Array and Structure [4 marks]

Array: A collection of elements of the *same* data type stored in contiguous memory locations, accessed via an index. Example: *int marks[5];*

Structure: A collection of elements of *different* data types grouped under one name, accessed via member names using the dot operator. Example: *struct Student { char name[20]; int roll; float gpa; };*

```
// Array
int arr[3] = {10, 20, 30};
printf("%d", arr[1]);       // 20

// Structure
struct Point { int x; float y; };
struct Point p = {3, 4.5};
printf("%d %.1f", p.x, p.y);  // 3  4.5
```

Q7(b). Differences between Structure and Union [3 marks]

Structure: each member has its own memory location; total size = sum of all members. **Union**: all members share the *same* memory location; total size = size of the largest member. Only one member can hold a valid value at a time in a union.

```
struct S { int i; float f; };   // size = 4 + 4 = 8 bytes
union  U { int i; float f; };   // size = 4 bytes (largest member)

struct S s; s.i = 5; s.f = 3.2;  // both valid simultaneously
union  U u; u.i = 5;              // u.f is now undefined
```

Q7(c). Program: 10 students – name, roll, marks, average [8 marks]

```
#include <stdio.h>
struct Student {
    char name[30];
    int  roll;
    float marks;
};

int main() {
    struct Student s[10];
    float total = 0;

    for (int i = 0; i < 10; i++) {
        printf("Enter name, roll, marks for student %d: ", i+1);
        scanf("%s %d %f", s[i].name, &s[i].roll, &s[i].marks);
        total += s[i].marks;
    }

    printf("\n%-20s %-8s %-8s\n", "Name", "Roll", "Marks");
    printf("-----------------------------------\n");
    for (int i = 0; i < 10; i++)
        printf("%-20s %-8d %.2f\n", s[i].name, s[i].roll, s[i].marks);

    printf("\nAverage Marks = %.2f\n", total / 10);
    return 0;
```

```
}
```

Q8(a). Program to check Even or Odd [5 marks]

```
#include <stdio.h>
int main() {
    int n;
    printf("Enter a number: ");
    scanf("%d", &n);
    if (n % 2 == 0)
        printf("%d is Even\n", n);
    else
        printf("%d is Odd\n", n);
    return 0;
}
```

Q8(b). Program to check Prime or Not [5 marks]

```
#include <stdio.h>
#include <math.h>
int main() {
    int n, i, prime = 1;
    printf("Enter a number: ");
    scanf("%d", &n);
    if (n < 2) prime = 0;
    for (i = 2; i <= (int)sqrt(n); i++) {
        if (n % i == 0) { prime = 0; break; }
    }
    printf("%d is %s\n", n, prime ? "Prime" : "Not Prime");
    return 0;
}
```

Q8(c). Program to add all even numbers from an array [5 marks]

```
#include <stdio.h>
int main() {
    int a[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    int n = sizeof(a) / sizeof(a[0]);
    int sum = 0;
    for (int i = 0; i < n; i++)
        if (a[i] % 2 == 0)
            sum += a[i];
    printf("Sum of even numbers = %d\n", sum);  // 30
    return 0;
}
```

Q9(a). What is an Operating System? [4 marks]

An **Operating System (OS)** is system software that acts as an intermediary between computer hardware and users/application programs. It manages hardware resources (CPU, memory, I/O devices) and provides a convenient environment for running programs. Examples: Windows, Linux, macOS, Android.

Q9(b). Functions of an Operating System [4 marks]

• **Process Management** – creates, schedules, and terminates processes; handles CPU scheduling.

• **Memory Management** – allocates and deallocates RAM; manages virtual memory and paging.

• **File System Management** – organises files/directories; controls read/write permissions.

• **Device Management** – manages I/O devices via device drivers.

• **Security & Protection** – controls user authentication and resource access.

• **User Interface** – provides CLI (command line) or GUI for user interaction.

Q9(c). What is Booting? [3 marks]

Booting is the process of starting or restarting a computer and loading the operating system into main memory (RAM) from storage. **Cold boot (hard boot)** – powering on from an off state. **Warm boot (soft boot)** – restarting a running system (e.g., Ctrl+Alt+Del). The BIOS/UEFI firmware initiates POST (Power-On Self Test) then loads the bootloader, which loads the OS kernel.

Q9(d). Differences between Compiler and Interpreter [4 marks]

Compiler vs Interpreter:

• **Translation:** Compiler — Translates entire source code at once; Interpreter — Translates line by line

• **Speed:** Compiler — Faster execution (compiled binary); Interpreter — Slower (translates at runtime)

• **Error detection:** Compiler — All errors shown after full scan; Interpreter — Stops at first error found

• **Output:** Compiler — Produces standalone executable; Interpreter — No separate output file

• **Examples:** Compiler — C, C++, Java (javac); Interpreter — Python, Ruby, JavaScript (interpreted mode)

Q10(a). Program to print diagonal of a 5×5 matrix [5 marks]

```
#include <stdio.h>
int main() {
    int a[5][5], i, j;
    printf("Enter 25 elements:\n");
    for (i = 0; i < 5; i++)
        for (j = 0; j < 5; j++)
            scanf("%d", &a[i][j]);

    printf("Main (left) diagonal: ");
    for (i = 0; i < 5; i++)
        printf("%d ", a[i][i]);

    printf("\nAnti (right) diagonal: ");
    for (i = 0; i < 5; i++)
        printf("%d ", a[i][4-i]);

    printf("\nDiagonal matrix (non-diagonal elements set to 0):\n");
    for (i = 0; i < 5; i++) {
        for (j = 0; j < 5; j++)
```

```
            printf("%4d", (i == j) ? a[i][j] : 0);
        printf("\n");
    }
    return 0;
}
```

Q10(b). Concatenate two strings without strcat() [5 marks]

```
#include <stdio.h>
int main() {
    char s1[100], s2[50];
    printf("Enter first string : "); scanf("%s", s1);
    printf("Enter second string: "); scanf("%s", s2);

    int i = 0, j = 0;
    while (s1[i] != '\0') i++;   // go to end of s1
    while (s2[j] != '\0')
        s1[i++] = s2[j++];        // copy s2 into s1
    s1[i] = '\0';                 // null-terminate

    printf("Concatenated: %s\n", s1);
    return 0;
}
```

Q10(c). Program to sort n elements (Bubble Sort) [5 marks]

```
#include <stdio.h>
int main() {
    int n, a[100], i, j, temp;
    printf("Enter n: "); scanf("%d", &n);
    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) scanf("%d", &a[i]);

    // Bubble Sort
    for (i = 0; i < n-1; i++)
        for (j = 0; j < n-i-1; j++)
            if (a[j] > a[j+1]) {
                temp   = a[j];
                a[j]   = a[j+1];
                a[j+1] = temp;
            }

    printf("Sorted array: ");
    for (i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

Q11(a). Convert Binary to Decimal: (1101.01)■ [4 marks]

Integer part: 1101

```
1 × 2³  =  1 × 8  =  8
1 × 2²  =  1 × 4  =  4
0 × 2¹  =  0 × 2  =  0
1 × 2■  =  1 × 1  =  1
                   ------
Integer sum        = 13
```

Fractional part: .01

```
0 × 2■⁻¹ =  0 × 0.5   = 0.00
1 × 2■⁻²  =  1 × 0.25  = 0.25
                         -----
Fractional sum          = 0.25
```

Result: (1101.01)■ = (13.25)■■

Q11(b). 2's Complement – definition and example [4 marks]

2's Complement is a method to represent negative numbers in binary. To find 2's complement of a number: Step 1 – find 1's complement (invert all bits); Step 2 – add 1 to the result.

```
Example: 2's complement of 0101 (decimal 5)
Step 1 – 1's complement: 1010
Step 2 – Add 1:          1010
                        +   1
                        ------
         2's complement: 1011  (represents -5 in 4-bit signed)
```

Q11(c). Subtraction using 2's Complement: 10101 – 00111 [4 marks]

We compute **10101** – **00111** as **10101 + (2's complement of 00111)**

```
Step 1: 1's complement of 00111 = 11000
Step 2: Add 1                   = 11001   (2's complement of 00111)

Step 3: Add
    10101
  + 11001
  -------
  101110   (6 bits; carry out of MSB is discarded for same-length result)

Discard carry bit → 01110 = (14)■■

Verification: 10101 = 21, 00111 = 7,  21 - 7 = 14 ✓
```

Q11(d). Convert Decimal to Octal: (5673)■■ [3 marks]

```
5673 ÷ 8 = 709  remainder 1
 709 ÷ 8 =  88  remainder 5
  88 ÷ 8 =  11  remainder 0
  11 ÷ 8 =   1  remainder 3
   1 ÷ 8 =   0  remainder 1

Reading remainders bottom to top: (5673)■■ = (13051)■
```

Answer: (5673)■■ = (13051)■

*** END OF ANSWER KEY — ESCS201 Programming for Problem Solving 2022-23 ***